

EvoMemBench: Benchmarking Agent Memory from a Self-Evolving Perspective

Yuyao Wang^{1*}, Zhongjian Zhang^{3*}, Mo Chi^{1*}, Kaichi Yu^{4*},

Yuhan Li¹, Miao Peng¹, Bing Tong^{1,2}, Chen Zhang², Yan Zhou², Jia Li^{1†}

¹Hong Kong University of Science and Technology (Guangzhou), ²Createlink Technology,

³Beijing University of Posts and Telecommunications, ⁴Beijing Institute of Technology

✉ Primary contact: jialeeli@hkust-gz.edu.cn

Abstract

Recent benchmarks for Large Language Model (LLM) agents mainly evaluate reasoning, planning, and execution. However, memory is also essential for agents, as it enables them to store, update, and retrieve information over time. This ability remains under-evaluated, largely because existing benchmarks do not provide a systematic way to assess memory mechanisms. In this paper, we study agent memory from a self-evolving perspective and introduce EvoMemBench, a unified benchmark organized along two axes: memory scope (in-episode vs. cross-episode) and memory content (knowledge-oriented vs. execution-oriented). We compare 15 representative memory methods with strong long-context baselines under a standardized protocol. Results show that current memory systems are still far from a general solution: long-context baselines remain highly competitive, memory helps most when the current context is insufficient or tasks are difficult, and no single memory form works consistently across all settings. Retrieval-based methods remain strong for knowledge-intensive settings, whereas procedural and long-term memory methods are more effective for execution-oriented tasks when their stored experience matches the task structure. We hope EvoMemBench facilitates future research on more effective memory systems for LLM-based agents. Our code is available at <https://github.com/DSAIL-Memory/EvoMemBench>.

1 Introduction

Large language models (LLMs) have enabled increasingly capable agents that reason, plan, and act through interaction with environments and external tools [7]. However, their backbone LLMs remain essentially stateless: they can condition on the current input context, but do not natively maintain or update persistent internal state across interactions. A common workaround is to append interaction histories, observations, and tool outputs to the prompt [38, 30], but this approach does not scale as histories grow beyond the finite context window. As a result, effective agent memory must go beyond storing past information by continuously updating, organizing, and reusing information as agents interact within and across tasks [10, 12].

We study this capability as *self-evolving memory*. From this perspective, memory should support two complementary forms of adaptation. Within an episode, agents must retain and revise evolving knowledge, as well as maintain task-relevant execution state during multi-step interaction. Across episodes, agents should accumulate reusable knowledge and distill execution experience that improves future decisions [11]. This yields a unified evaluation view along two axes: *in-episode* vs. *cross-episode* evolution, and *knowledge-oriented* vs. *execution-oriented* memory.

*Equal contribution

†Corresponding Author.

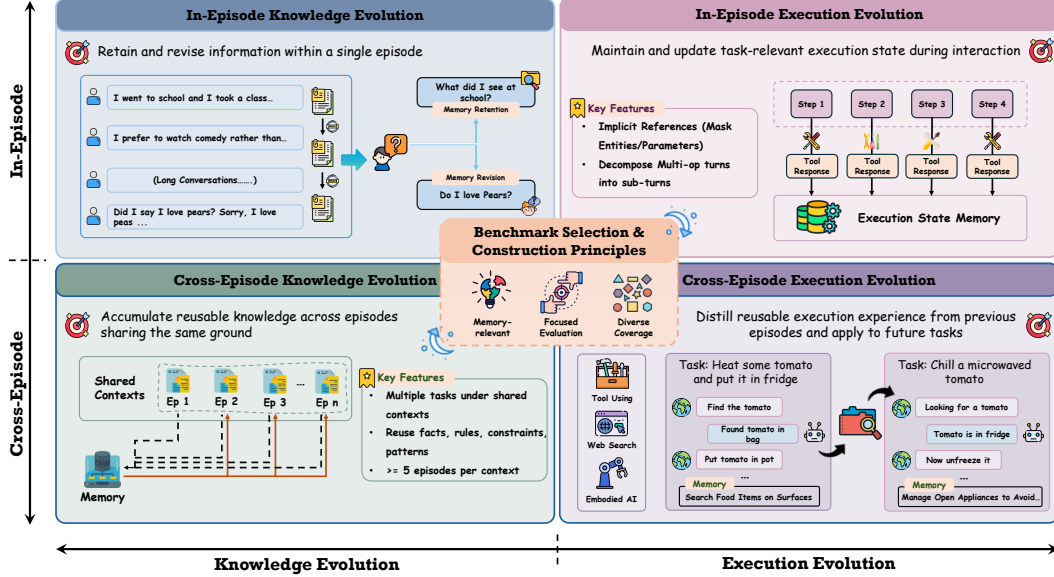


Figure 1: Overview of EvoMemBench.

Existing memory benchmarks cover only parts of this space. Text-centric benchmarks such as LoCoMo [22], LongMemEval [35], and MemoryAgentBench [17] mainly evaluate knowledge retention, retrieval, or revision in conversational or document-like contexts, but do not test whether memory supports action and tool-based execution. Recent agentic or lifelong memory benchmarks, including MemoryArena [15], Evo-Memory [34], StuLife [3], and MemoryBench [1], move closer to interactive or cross-episode settings, but still do not jointly evaluate in-episode and cross-episode evolution while separating knowledge-oriented and execution-oriented memory demands. Moreover, some closely related benchmarks [34, 1] are not fully open-sourced, limiting reproducible comparison.

To address these gaps, we introduce EvoMemBench, a benchmark for evaluating agent memory from a self-evolving perspective. EvoMemBench covers four settings: in-episode knowledge evolution, in-episode execution evolution, cross-episode knowledge evolution, and cross-episode execution evolution. These settings provide a unified testbed for assessing whether memory systems can update and reuse information over time. Our contributions are summarized as follows:

- We benchmark agent memory from a self-evolving perspective, organizing memory evaluation along the axes of **in-episode vs. cross-episode** evolution and **knowledge vs. execution** demands.
- We introduce EvoMemBench, a unified and open benchmark that comprehensively evaluates existing memory systems across four forms of self-evolving memory.
- We provide a standardized evaluation protocol and comparative analysis of representative memory methods, revealing when memory helps, when it hurts, and which memory forms best match different agent-memory demands.

2 Related Work

Memory mechanisms for LLM agents. Memory is an important component for LLM-based agents, as it allows agents to keep useful information beyond the current context window and reuse past experience in later tasks. Existing methods mainly differ in what they store and how memory is used. Retrieval-based methods store past information in external indexes and retrieve relevant entries during inference [28, 42, 9, 29, 8, 19, 2]. Long-term memory systems further introduce structured storage, updating, and hierarchical management for persistent facts, preferences, and interaction histories [5, 37, 20, 18, 23]. Procedural memory methods instead abstract trajectories into reusable workflows, skills, or reasoning strategies [33, 43, 32, 25, 14]. Recent studies also explore evolving contexts or memory architectures, where the memory itself is updated or adapted over time [41, 40, 44]. Together, these methods show that agent memory is moving from passive storage toward reusable and evolving experience. However, they are usually evaluated on different

Table 1: Comparison between existing memory benchmarks and EvoMemBench.

Benchmark	In-Episode		Cross-Episode		Open-source
	Knowledge Evolution	Execution Evolution	Knowledge Evolution	Execution Evolution	
LoCoMo [22]	✓	✗	✗	✗	✓
LongMemEval [35]	✓	✗	✗	✗	✓
MemoryAgentBench [17]	✓	✗	✗	✗	✓
StuLife [3]	✗	✓	✓	✓	✓
MemoryBench [1]	✓	✗	✓	✗	✗
Evo-Memory [34]	✗	✗	✓	✓	✗
MemoryArena [15]	✓	✓	✗	✗	✓
EvoMemBench (ours)	✓	✓	✓	✓	✓

tasks and protocols, making it hard to compare which memory mechanisms are useful for different self-evolving needs.

Memory benchmarks for LLMs and agents. Existing memory benchmarks cover important but partial aspects of agent memory. Text-based benchmarks, such as LoCoMo and LongMemEval, evaluate long-term conversational memory through question answering over multi-session histories [22, 35], while MemoryAgentBench studies memory agents under incremental multi-turn inputs and evaluates abilities such as retrieval, test-time learning, long-range understanding, and selective forgetting [17]. More recent benchmarks move toward agentic and continual memory: MemoryBench studies continual learning from simulated user feedback with declarative and procedural memory [1], Evo-Memory evaluates test-time memory evolution over sequential task streams [34], and MemoryArena examines memory-guided action in multi-session environments [15]. However, as shown in Table 1, existing memory benchmarks do not offer a unified view of how memory evolves across knowledge and execution, within and across episodes.

3 Problem Formulation

3.1 Agents with Memory Mechanisms

We consider an LLM-based agent that interacts with an environment over a sequence of task episodes, where each episode corresponds to one complete attempt to solve a task instance. At step t of episode e , the agent receives an observation o_t^e and has access to the interaction history $h_{t-1}^e = (o_1^e, a_1^e, \dots, o_{t-1}^e, a_{t-1}^e)$. It then takes an action a_t^e and receives feedback y_t^e . For example, in a tool-use task, the action may be a tool call and the feedback is the returned result. As the episode proceeds, the interaction history grows and may exceed the finite context window of the LLM. The agent therefore maintains a memory state $m_t^e \in \mathcal{M}$, which serves as a compact representation of useful information from past interactions. At step t , the agent conditions its action on both the current interaction context and the memory state: $a_t^e \sim \pi(\cdot \mid h_{t-1}^e, o_t^e, m_t^e)$, where π denotes the agent policy. After receiving feedback y_t^e , the memory state is updated as $m_{t+1}^e = \mathcal{U}(m_t^e, o_t^e, a_t^e, y_t^e)$, where $\mathcal{U}$ denotes the memory update function.

3.2 Scope of Memory Evolution: In-Episode and Cross-Episode

In-episode memory. Memory is maintained only within a single episode. It summarizes useful information from earlier steps of that episode and is used to support later decisions within the same episode. Thus, in-episode memory is a task-level compact state that retains information from the current episode for subsequent actions.

Cross-episode memory. Memory is maintained across different episodes. It summarizes useful information from previous task attempts and is used to support decisions in later episodes. Thus, cross-episode memory is an experience-level compact state that retains reusable information from past episodes while avoiding the direct reuse of episode-specific details that are no longer relevant.

3.3 Content of Memory Evolution: Knowledge and Execution

Knowledge evolution. Knowledge evolution refers to maintaining information that supports later reasoning or answering. Such information may include facts, constraints, preferences, domain rules, or task patterns. As new information arrives, memory should preserve valid earlier information and

update outdated or contradicted information when later evidence changes what should be remembered. Thus, knowledge evolution concerns both retaining useful information and revising memory when stored information becomes inaccurate.

Execution evolution. Execution evolution refers to the maintenance of information that supports action selection and task progress. Such information can include task progress, intermediate results, tool outputs, unresolved subgoals, previous decisions, action routines, and task-solving procedures. As interaction proceeds, memory should help the agent keep track of what has been done, what remains to be done, and which actions are likely to be useful next. Thus, execution evolution is concerned with maintaining action-relevant state and reusable execution experience.

4 Benchmark Construction

Based on the above forms of memory evolution, we select or reconstruct six datasets to evaluate different aspects of agent memory, as summarized in Table 2. Our dataset selection follows three criteria: *memory relevance*, *focused evaluation*, and *coverage*. First, the tasks should leave room for explicit memory to improve the performance of strong LLMs. Second, they should involve clear memory dependencies and provide verifiable outcomes. Third, they should cover diverse domains and task formats to reduce dataset-specific bias. For clarity, we refer to the six resulting benchmarks as INEP-KNOW, INEP-EXEC, CROSSEP-KNOW, CROSSEP-TOOL, CROSSEP-WEB, and CROSSEP-EMB.

4.1 In-Episode Knowledge Evolution

INEP-KNOW. This dataset evaluates whether an agent can retain and revise information within a single episode. We build it from the Accurate Retrieval and Selective Forgetting subsets of MemoryAgentBench. These subsets are organized in an incremental multi-turn format, where information is provided over time and the agent must update its memory during the episode.

4.2 In-Episode Execution Evolution

INEP-EXEC. This dataset evaluates whether an agent can maintain task-relevant execution state during an ongoing tool-use interaction. It is built from BFCL-Multiturn-LongContext [26], which contains multi-turn tool-use tasks with noisy context. We make the dataset more memory-oriented through two reconstruction steps. First, we replace explicit mentions of entities, parameters, or results with implicit references when they can be inferred from previous turns, forcing the agent to recover them from memory. Second, we split turns with multiple dependent ground-truth actions into simpler sub-turns while preserving the original action order, so that later sub-turns can refer to earlier objects or states. The reconstruction is assisted by GPT-5-mini with our designed prompts, and the correctness of the reconstructed samples is manually verified. This reconstruction reduces the influence of complex planning and emphasizes execution-state tracking across turns. Detailed prompts are provided in Appendix B.

4.3 Cross-Episode Knowledge Evolution

CROSSEP-KNOW. This dataset evaluates whether an agent can accumulate reusable knowledge across episodes with a shared background. We build it from CL-Bench [6], where each context provides new knowledge for multiple context-dependent tasks. CROSSEP-KNOW covers four knowledge forms: domain facts, rule systems, procedures, and empirical patterns. Because the required knowledge is provided in the context, the dataset reduces reliance on pre-trained knowledge and focuses on memory accumulation. We group CL-Bench samples by shared context and keep only contexts with at least five samples, yielding 120 contexts and 884 samples. We further split these contexts into Easy, Medium, and Hard subsets by tertiles according to the DeepSeek-V3.2 baseline score. For each context, episodes are processed sequentially: memory is updated after each episode and used in later episodes from the same context.

Table 2: Dataset statistics in EvoMemBench, where #Samples denotes evaluation samples.

Setting	Dataset	Source	Subset / Domain (#Samples)
In-Episode Knowledge	INEP-KNOW	MemoryAgentBench	Accurate Retrieval (2000); Selective Forgetting (800)
In-Episode Execution	INEP-EXEC	BFCL-MultiTurn-LongContext	Gorilla File System (200); Vehicle Control (200); Trading Bots (200); Travel Booking (200)
Cross-Episode Knowledge	CROSSEP-KNOW	CL-Bench	Domain Knowledge Reasoning (294); Rule System Application (257); Procedural Task Execution (306); Empirical Discovery & Simulation (27)
	CROSSEP-TOOL	BFCL-MultiTurn-Base	Gorilla File System (200); Vehicle Control (200); Trading Bots (200); Travel Booking (200)
Cross-Episode Execution	CROSSEP-WEB	xbench-DeepSearch	Deep Search (100)
		WebWalkerQA	Web QA (170)
	CROSSEP-EMB	ALFWorld	Examine in Light (19); Pick&Place (46); Clean&Place (37); Cool&Place (28); Heat&Place (25); Pick Two&Place (45)

4.4 Cross-Episode Execution Evolution

Cross-episode execution evolution evaluates whether an agent can learn reusable execution experience from previous episodes and apply it to future tasks. We instantiate this setting with three datasets covering tool use, web search, and embodied interaction. For each dataset, memory is accumulated and evaluated within each sub-dataset, so that the evaluation focuses on domain-specific execution experience rather than mixing heterogeneous environments.

CROSSEP-TOOL. We reconstruct BFCL-Multiturn-Base following the same procedure as INEP-EXEC, and evaluate memory across its four categories. This dataset tests whether memory can capture reusable tool-use patterns, API constraints, and parameter-filling procedures.

CROSSEP-WEB. We use xbench-DeepSearch [4] and WebWalkerQA [36] to evaluate memory in web search tasks. Following MemEvolve [40], we use 170 sampled examples from WebWalkerQA. This dataset tests whether agents can reuse search and verification experience from previous trajectories.

CROSSEP-EMB. We use the ALFWorld [31] test set and evaluate memory across its six task categories. This dataset tests whether memory can retain action routines and environment-specific information for later embodied tasks.

5 Experiments

5.1 Experiment Setup

Methods. We evaluate memory-free and memory-augmented agents. For memory-free agents, we use Gemini-3-Flash [13], GPT-5-mini [24], and DeepSeek-V3.2 [21]. For memory-augmented agents, we use DeepSeek-V3.2 as the unified backbone and compare five categories of memory methods: retrieval-augmented memory (BM25 [28], Qwen3-Emb-4B [42], GraphRAG [9]), short-term memory (MemAgent [39], MemoBrain [27]), general long-term memory (Mem0 [5], A-MEM [37], MemOS [20], MemoryOS [18]), procedural long-term memory (AWM [33], SkillWeaver [43], AgentKB [32], ACE [41], ReasoningBank [25]), and meta-evolution memory (MemEvolve [40]). Full implementation details and applicability criteria are provided in Appendix A.3.

Metrics. We evaluate both task performance and efficiency. For knowledge-oriented tasks (INEP-KNOW and CROSSEP-KNOW), we report *answer accuracy*, which measures whether the model produces the correct answer. For execution-oriented tasks (INEP-EXEC, CROSSEP-TOOL, CROSSEP-WEB, and CROSSEP-EMB), we report *success rate*, which measures whether the agent completes the final goal. We also report *token usage* as an efficiency metric. *Token usage* measures the total number of LLM inference tokens, including both input and output tokens used by the agent and the memory module. When comparing methods across multiple subsets, we also report average rank, where a lower rank indicates better overall performance.

Table 3: Answer accuracy (%) and ranking of different memory methods on INEP-KNOW. Lower ranks indicate better performance.

Category	Method	Memory Retention					Memory Revision			Overall Rank
		Event QA	LME (S*)	Ruler qa1	Ruler qa2	Avg. Rank	FC MH	FC SH	Avg. Rank	
Long-Context LLM	Gemini-3-Flash	98.20	83.00	94.00	77.00	1.00	58.00	96.00	1.00	1.00
	GPT-5-mini	80.60	62.33	85.00	73.00	3.50	24.00	77.00	2.00	3.00
	DeepSeek-V3.2	83.00	32.33	64.00	42.00	6.75	10.00	65.00	3.00	5.50
Retrieval-Augmented Memory	BM25	88.80	50.33	<u>73.00</u>	<u>55.00</u>	4.25	7.00	54.00	4.00	4.17
	Qwen3-Emb-4B	83.60	21.33	38.00	38.00	8.50	4.00	30.00	7.50	8.17
	GraphRAG	74.20	22.00	31.00	31.00	10.50	4.00	18.00	9.50	10.17
Short-Term Memory	MemAgent	71.80	12.67	26.00	39.00	10.75	3.00	19.00	10.00	10.50
	MemoBrain	77.60	10.67	30.00	32.00	11.00	1.00	20.00	11.00	11.00
General Long-Term Memory	Mem0	79.20	52.00	75.00	68.00	<u>4.50</u>	7.00	<u>49.00</u>	<u>4.50</u>	<u>4.50</u>
	A-MEM	91.20	<u>51.00</u>	60.00	46.00	4.25	<u>5.00</u>	35.00	6.00	4.83
	MemOS	<u>90.40</u>	41.33	55.00	43.00	5.50	3.00	35.00	7.50	6.17
	MemoryOS	84.80	38.67	41.00	37.00	7.50	2.00	30.00	9.50	8.17

Implementation. For each dataset, we reuse the inference pipeline or agent framework from the source benchmark whenever possible, and only replace the LLM backbone or attach a memory module. Memory is initialized, updated, and reset according to the evaluation setting. In in-episode settings, memory is initialized at the beginning of each episode, updated online, and cleared after the episode ends. Specifically, memory is updated after each progressive information block in INEP-KNOW and after each interaction turn in INEP-EXEC. For INEP-EXEC, we evaluate four context budgets: 16K, 32K, 64K, and 128K, since memory content also consumes the available context window. In cross-episode settings, memory is updated after each episode and reused in subsequent episodes from the same context or dataset subset. For cross-episode execution setting, we further evaluate cross-environment transfer, where memory is built on a source subset and kept fixed during evaluation on a target subset. More implementation details are provided in Appendix B.2.

5.2 Main Results

5.2.1 In-Episode Knowledge Evolution

Finding 1: Strong long-context baselines remain highly competitive. As shown in Table 3 and Figure 2, direct access to the raw context remains a strong baseline. Gemini-3-Flash achieves the best rank on both retention and revision, suggesting that when the context budget is sufficient, preserving the original evidence may be more reliable than relying on an external memory abstraction.

Finding 2: Existing memory methods are better at retention than revision. As shown in Table 3, explicit memory methods show clearer benefits on retention-oriented tasks than on revision-oriented tasks. BM25, Mem0, and

A-MEM improve over the weaker no-memory baseline on retention subsets, but their performance drops sharply on FactConsolidation, especially in the multi-hop setting. This pattern suggests that the main bottleneck is not merely retrieving relevant evidence. The harder challenge is deciding which stored information should be updated, suppressed, or replaced when later evidence conflicts with earlier evidence. Thus, in-episode knowledge evolution requires both effective memory access and robust memory revision.

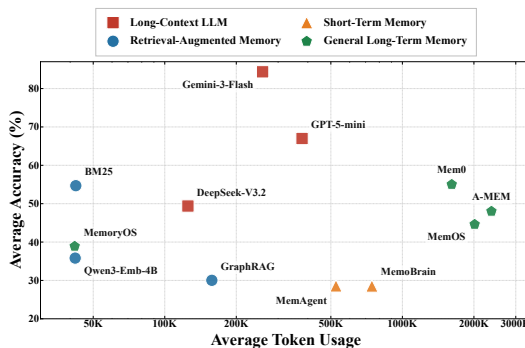


Figure 2: Accuracy and cost on INEP-KNOW.

Table 4: Success rate (%) comparison on INEP-EXEC under different context lengths. Best results among memory methods are bolded, and runner-up results are underlined.

Method	Gorilla File System				Vehicle Control				Trading Bots				Travel Booking				Overall			
	16K	32K	64K	128K	16K	32K	64K	128K	16K	32K	64K	128K	16K	32K	64K	128K	16K	32K	64K	128K
<i>Long-Context LLM</i>																				
Gemini-3-Flash	54.0	50.0	52.0	52.0	60.0	70.0	77.6	60.0	28.0	80.0	78.0	80.0	60.0	62.0	60.0	56.0	50.5	65.5	66.9	62.0
GPT-5-mini	34.0	34.0	40.0	34.0	46.0	44.0	63.7	44.0	20.0	62.0	62.0	56.0	40.0	30.0	32.0	36.0	35.0	42.5	49.4	42.5
DeepSeek-V3.2	52.0	46.0	46.0	48.0	38.0	40.0	73.2	44.0	8.0	40.0	46.0	46.0	16.0	16.0	16.0	20.0	28.5	35.5	45.3	39.5
<i>Retrieval-Augmented Memory</i>																				
BM25	50.0	<u>56.0</u>	48.0	54.0	<u>50.0</u>	46.0	72.8	40.0	<u>20.0</u>	<u>54.0</u>	48.0	42.0	26.0	22.0	20.0	26.0	36.5	<u>44.5</u>	47.2	40.5
Qwen3-Emb-4B	48.0	58.0	44.0	<u>58.0</u>	42.0	<u>48.0</u>	68.7	42.0	16.0	32.0	46.0	44.0	<u>32.0</u>	20.0	<u>26.0</u>	24.0	34.5	39.5	46.2	42.0
GraphRAG	<u>56.0</u>	<u>56.0</u>	<u>54.0</u>	<u>54.0</u>	48.0	38.0	80.2	44.0	12.0	24.0	46.0	42.0	24.0	<u>28.0</u>	28.0	24.0	35.0	36.5	<u>52.0</u>	41.0
<i>Short-Term Memory</i>																				
MemAgent	48.0	48.0	<u>54.0</u>	46.0	34.0	36.0	<u>79.8</u>	36.0	10.0	40.0	40.0	52.0	12.0	22.0	16.0	14.0	26.0	36.5	47.4	37.0
MemoBrain	40.0	46.0	<u>54.0</u>	46.0	40.0	32.0	38.0	42.0	4.0	32.0	50.0	42.0	16.0	12.0	20.0	26.0	25.0	30.5	40.5	39.0
<i>General Long-Term Memory</i>																				
Mem0	54.0	44.0	46.0	60.0	46.0	44.0	70.9	38.0	16.0	48.0	38.0	38.0	24.0	18.0	24.0	20.0	35.0	38.5	44.7	39.0
A-MEM	52.0	52.0	50.0	46.0	48.0	44.0	76.2	48.0	34.0	36.0	48.0	40.0	16.0	26.0	24.0	26.0	<u>37.5</u>	39.5	49.5	40.0
MemOS	52.0	52.0	<u>54.0</u>	50.0	44.0	36.0	79.0	38.0	12.0	40.0	48.0	46.0	22.0	12.0	14.0	16.0	32.5	35.0	48.8	37.5
MemoryOS	48.0	58.0	42.0	44.0	36.0	36.0	72.5	40.0	10.0	38.0	42.0	20.0	20.0	20.0	22.0	26.0	28.5	38.0	44.6	32.5
<i>Procedural Long-Term Memory</i>																				
AWM	50.0	46.0	48.0	48.0	46.0	46.0	78.3	32.0	10.0	58.0	72.0	42.0	20.0	22.0	14.0	18.0	31.5	43.0	53.1	35.0
SkillWeaver	48.0	50.0	48.0	54.0	44.0	42.0	73.8	46.0	14.0	42.0	58.0	46.0	30.0	<u>28.0</u>	<u>26.0</u>	<u>32.0</u>	34.0	40.5	51.5	<u>44.5</u>
AgentKB	46.0	50.0	<u>54.0</u>	50.0	8.0	40.0	77.2	56.0	18.0	26.0	56.0	<u>50.0</u>	18.0	8.0	10.0	14.0	22.5	31.0	49.3	42.5
ACE	52.0	42.0	56.0	54.0	42.0	34.0	78.4	44.0	8.0	42.0	44.0	48.0	26.0	22.0	24.0	20.0	32.0	35.0	50.6	41.5
ReasoningBank	62.0	52.0	50.0	54.0	60.0	52.0	74.6	<u>54.0</u>	16.0	44.0	36.0	48.0	34.0	50.0	28.0	36.0	43.0	49.5	47.1	48.0
<i>Meta-Evolution Memory</i>																				
MemEvolve	52.0	48.0	46.0	54.0	42.0	44.0	74.8	50.0	<u>20.0</u>	50.0	<u>62.0</u>	46.0	30.0	22.0	20.0	22.0	36.0	41.0	50.7	43.0

5.2.2 In-Episode Execution Evolution

Finding 3: Memory helps more when the context budget is constrained. As reported in Table 4, compared with the DeepSeek-V3.2 backbone, the best-performing memory method improves the overall success rate by +14.5 points at 16K, +14.0 points at 32K, +7.8 points at 64K, and +8.5 points at 128K. The largest gains appear at 16K and 32K, where the raw interaction history is more likely to be truncated. In this regime, memory can compensate for missing history by preserving useful states or execution patterns. However, as the context budget increases, the marginal benefit of memory becomes smaller and less stable. At 128K, several memory methods even fall below the no-memory baseline, indicating that appended memory can introduce noise or consume useful context space without adding sufficient information.

Finding 4: Execution tasks benefit from procedural guidance, but compression is risky. As reported in Table 4, procedural long-term memory methods show the strongest overall pattern. ReasoningBank achieves the best overall score at 16K, 32K, and 128K, with 43.0, 49.5, and 48.0, respectively, while AWM performs best at 64K with 53.1. This suggests that tool-use episodes benefit from reusable execution guidance, such as action routines, failure-aware strategies, or parameter-filling procedures. At the same time, short-term compression is not always safe. MemAgent and MemoBrain achieve overall success rates of only 26.0 and 25.0 at 16K, both below DeepSeek-V3.2’s 28.5. At 32K, MemoBrain remains below the baseline (30.5 vs. 35.5), and at 128K, MemAgent also underperforms the baseline (37.0 vs. 39.5). This suggests that execution-state tracking requires fine-grained details, such as exact tool-call arguments, entity references, and intermediate results.

5.2.3 Cross-Episode Knowledge Evolution

Finding 5: Memory helps more on difficult tasks, but can hurt easy tasks. Cross-episode knowledge evolution does not show uniform gains from memory. As reported in Table 5, on the overall easy split, the memory-free DeepSeek-V3.2 achieves 52.1, while the best memory method, Qwen3-Emb-4B, reaches only 50.0. General long-term memory methods perform even lower, ranging from 43.8 to 46.8. In contrast, memory becomes more useful on harder tasks. On the overall hard split, DeepSeek-V3.2 obtains 0.0, while ACE improves the score to 13.0, followed by BM25 with 10.1 and MemoryOS with 9.0. This suggests that when the current context is already sufficient for the

Table 5: Answer accuracy (%) comparison on CROSSEP-KNOW under different difficulty levels. Best results are bolded, and runner-up results among memory methods are underlined.

Category	Method	Domain Knowledge Reasoning			Empirical Discovery & Simulation			Procedural Task Execution			Rule System Application			Overall		
		Easy	Medium	Hard	Easy	Medium	Hard	Easy	Medium	Hard	Easy	Medium	Hard	Easy	Medium	Hard
Long-Context LLM	Gemini-3-Flash	44.4	17.9	9.1	0.0	0.0	11.1	42.9	12.6	5.0	61.8	21.2	6.0	37.3	13.0	7.8
	GPT-5-mini	61.7	37.6	16.5	50.0	0.0	22.2	54.7	22.3	8.8	76.7	26.6	9.9	60.8	21.6	14.4
	DeepSeek-V3.2	64.6	21.4	0.0	10.0	0.0	0.0	57.9	10.0	0.0	75.7	18.8	0.0	52.1	12.6	0.0
Retrieval-Augmented Memory	BM25	60.9	23.5	4.8	0.0	0.0	<u>22.2</u>	54.5	13.2	12.1	<u>73.5</u>	18.4	1.4	47.2	13.8	<u>10.1</u>
	Qwen3-Emb-4B	<u>63.5</u>	23.7	6.7	10.0	0.0	11.1	56.3	13.0	8.1	70.3	18.3	5.9	50.0	13.7	7.9
	GraphRAG	60.4	<u>25.4</u>	<u>7.6</u>	0.0	0.0	11.1	<u>59.2</u>	13.4	9.2	72.4	15.9	2.8	48.0	13.7	7.7
General Long-Term Memory	Mem0	55.3	15.9	5.1	0.0	0.0	0.0	50.5	12.5	4.4	72.6	18.4	5.8	44.6	11.7	3.8
	A-MEM	49.4	13.8	2.9	10.0	0.0	11.1	50.4	16.1	7.4	65.3	10.1	5.5	43.8	10.0	6.7
	MemOS	55.7	<u>25.4</u>	4.8	10.0	0.0	11.1	57.5	16.3	7.4	63.9	17.3	4.0	46.8	14.8	6.8
	MemoryOS	51.5	19.0	1.9	10.0	0.0	<u>22.2</u>	52.7	12.4	6.6	62.7	12.7	5.4	44.2	11.0	9.0
Procedural Long-Term Memory	AWM	54.2	22.7	9.0	0.0	0.0	0.0	60.9	12.8	6.9	68.8	14.4	3.3	46.0	12.5	4.8
	SkillWeaver	64.4	23.5	4.8	10.0	0.0	11.1	58.8	7.1	2.2	60.5	14.8	4.5	<u>48.4</u>	11.3	5.6
	AgentKB	56.9	22.1	5.1	10.0	0.0	0.0	51.2	10.3	5.5	64.5	12.7	3.7	45.6	11.3	3.6
	ACE	58.0	24.6	2.9	0.0	0.0	33.3	46.2	<u>17.2</u>	8.1	74.9	24.0	7.8	44.8	16.5	13.0
	ReasoningBank	60.9	27.5	6.1	0.0	0.0	0.0	48.0	18.9	11.0	62.1	16.5	<u>7.4</u>	42.7	<u>15.7</u>	6.1
Meta-Evolution Memory	MemEvolve	54.3	16.7	3.8	10.0	0.0	11.1	47.3	12.4	<u>11.4</u>	72.5	<u>18.9</u>	3.1	46.0	12.0	7.4

Table 6: Success rate (%) comparison across CROSSEP-TOOL, CROSSEP-WEB and CROSSEP-EMB. Best results among memory methods are bolded, and runner-up results are underlined.

Method	Tool Using					Web Search			Embodied AI							Overall Rank
	Gorilla File System	Vehicle Control	Trading Bots	Travel Booking	Avg. Rank	xbench -DS	WebWalker QA	Avg. Rank	Examine in Light	Pick &Place	Clean &Place	Cool &Place	Heat &Place	Pick Two &Place	Avg. Rank	
Long-Context LLM																
Gemini-3-Flash	74.0	62.0	88.0	54.0	3.0	76.0	72.4	5.0	68.4	76.1	43.2	64.3	60.0	26.7	12.0	7.8
GPT-5-mini	50.0	50.0	72.0	44.0	9.5	72.0	71.8	10.0	36.8	69.6	21.6	25.0	40.0	40.0	15.5	12.7
DeepSeek-V3.2	64.0	36.0	54.0	26.0	13.5	74.0	61.2	14.0	73.7	93.5	56.8	60.7	56.0	57.8	10.0	11.8
Retrieval-Augmented Memory																
BM25	68.0	88.0	66.0	32.0	5.8	79.0	65.9	6.5	68.4	97.8	62.2	60.7	64.0	71.1	7.8	6.9
Qwen3-Emb-4B	68.0	82.0	78.0	34.0	4.0	78.0	65.3	8.0	79.0	97.8	70.3	60.7	64.0	71.1	6.0	5.7
GraphRAG	66.0	68.0	72.0	40.0	6.5	73.0	70.0	11.0	63.2	97.8	75.7	60.7	72.0	77.8	5.8	6.9
General Long-Term Memory																
Mem0	76.0	58.0	48.0	30.0	8.5	82.0	70.6	4.0	79.0	89.1	67.6	57.1	68.0	53.3	8.2	7.9
A-MEM	68.0	62.0	70.0	26.0	8.8	76.0	71.8	6.0	84.2	91.9	81.1	64.3	80.0	84.4	1.8	5.2
MemOS	72.0	60.0	36.0	28.0	9.8	78.0	71.2	5.0	63.2	91.3	67.6	64.3	68.0	53.3	7.8	8.4
MemoryOS	72.0	72.0	56.0	36.0	5.8	78.0	69.4	6.5	73.7	95.7	75.7	64.3	60.0	57.8	6.5	6.5
Procedural Long-Term Memory																
AWM	64.0	56.0	74.0	24.0	11.0	76.0	72.4	5.0	73.7	91.3	78.4	71.4	76.0	64.4	4.3	7.2
SkillWeaver	66.0	48.0	44.0	28.0	12.3	74.0	64.7	13.5	63.2	84.8	59.5	53.6	64.0	66.7	10.2	12.2
AgentKB	62.0	71.4	74.0	30.0	8.3	79.0	68.8	6.0	68.4	89.1	67.6	85.7	80.0	73.3	4.2	6.5
ACE	68.0	56.0	14.0	34.0	9.8	77.0	65.3	9.5	84.2	78.3	75.7	21.4	76.0	82.2	4.8	8.2
ReasoningBank	64.0	82.0	86.0	42.0	4.8	76.0	75.3	4.5	79.0	87.0	81.1	67.9	60.0	73.3	4.5	5.4
Meta-Evolution Memory																
MemEvolve	66.0	70.0	66.0	28.0	8.8	73.0	65.3	13.0	57.9	89.1	64.9	78.6	76.0	75.6	6.0	8.8

backbone model, additional memory may introduce irrelevant evidence, mismatched past information, or unnecessary prompt bias.

Finding 6: The bottleneck is forming reusable knowledge, not only storing more information.

Table 5 shows that ACE achieves the best overall performance among memory methods on both medium and hard tasks, with 16.5 and 13.0, respectively. Its advantage is especially clear on rule system application, where it obtains 74.9, 24.0, and 7.8 on the easy, medium, and hard splits, outperforming all other memory methods at each difficulty level. This indicates that effective memory should support how knowledge is applied, rather than only what knowledge is stored.

5.2.4 Cross-Episode Execution Evolution

Finding 7: Different execution domains favor different memory forms. Table 6 shows that the best memory family changes across execution domains. In Tool Using, retrieval-augmented memory achieves the best family-level average rank among memory methods, with an average rank of 5.43. In Web Search, general long-term memory performs best, with an average rank of 5.38.



Figure 3: Cross-environment transfer results in CROSSEP-TOOL.

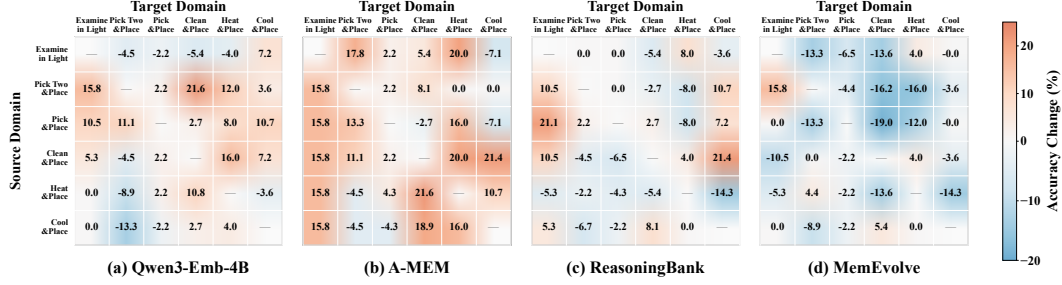


Figure 4: Cross-environment transfer results in CROSSEP-EMB.

In Embodied AI, procedural long-term memory performs best, with an average rank of 5.60. This result shows that cross-episode execution evolution cannot be addressed by one fixed memory form. Retrieval-augmented memory reuses similar past cases, which fits tool-use tasks where similar API calls and parameter patterns recur. General long-term memory maintains persistent information, which is useful in web search where evidence and verification habits must be accumulated. Procedural long-term memory stores strategies, workflows, or skills, which fits embodied tasks where action routines and affordances are important.

Finding 8: Transfer depends on whether memory matches the target decision process. Figure 3 indicates that tool-use tasks exhibit more stable positive transfer. This pattern may be related to the fact that different tool-use environments often involve similar decision steps, such as checking intermediate states, using tool outputs to choose the next call, and verifying the final state. Memory that preserves such decision procedures may therefore be more reusable across tool-use domains. In contrast, Figure 4 shows that transfer in ALFWorld is less stable and more likely to become neutral or negative. Compared with tool-use domains, ALFWorld sub-environments differ more substantially in their task requirements and execution conditions. As a result, experience accumulated in one embodied setting may not apply as consistently to another. Overall, these results suggest that cross-environment transfer depends not only on whether memory stores past execution experience, but also on whether that experience remains aligned with the target decision process.

5.2.5 Summary

These findings reveal a consistent pattern. (1) Current memory methods can improve agent performance, but not uniformly: strong memory-free long-context baselines remain highly competitive, and explicit memory provides the clearest gains only in some settings. (2) Memory helps most when the current context is insufficient, the task is more difficult, or the stored experience matches the target decision process; it can hurt when memory introduces irrelevant evidence, removes fine-grained execution details through compression, or transfers mismatched procedures across environments. (3) No single memory form is sufficient across all settings: retrieval remains a strong choice for knowledge-oriented demands, while procedural and long-term memory are more effective for execution-oriented problems when their stored form aligns with the reusable structure of the task. Overall, current methods still do not provide a general solution for self-evolving agent memory.

6 Conclusion

In this work, we introduce `EvoMemBench`, a benchmark for evaluating how agent memory evolves within and across episodes. `EvoMemBench` covers four settings by combining two axes: memory scope (in-episode vs. cross-episode) and memory content (knowledge-oriented vs. execution-oriented), providing a unified testbed for adaptive agent memory. Experiments show that current memory methods are still far from a general solution. We hope `EvoMemBench` will support future research on more effective memory systems for LLM-based agents.

References

- [1] Qingyao Ai, Yichen Tang, Changyue Wang, Jianming Long, Weihang Su, and Yiqun Liu. Memorybench: A benchmark for memory and continual learning in llm systems. *arXiv preprint arXiv:2510.17281*, 2025. URL <https://arxiv.org/abs/2510.17281>.
- [2] Petr Anokhin, Nikita Semenov, Artyom Sorokin, Dmitry Evseev, Andrey Kravchenko, Mikhail Burtsev, and Evgeny Burnaev. Arigraph: Learning knowledge graph world models with episodic memory for llm agents, 2025. URL <https://arxiv.org/abs/2407.04363>.
- [3] Yuxuan Cai, Yipeng Hao, Jie Zhou, Hang Yan, Zhikai Lei, Rui Zhen, Zhenhua Han, Yutao Yang, Junsong Li, Qianjun Pan, Tianyu Huai, Qin Chen, Xin Li, Kai Chen, Bo Zhang, Xipeng Qiu, and Liang He. Building self-evolving agents via experience-driven lifelong learning: A framework and benchmark. *arXiv preprint arXiv:2508.19005*, 2025. URL <https://arxiv.org/abs/2508.19005>.
- [4] Kaiyuan Chen, Yixin Ren, Yang Liu, Xiaobo Hu, Haotong Tian, Tianbao Xie, Fangfu Liu, Haoye Zhang, Hongzhang Liu, Yuan Gong, et al. xbench: Tracking agents productivity scaling with profession-aligned real-world evaluations. *arXiv preprint arXiv:2506.13651*, 2025.
- [5] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. In *ECAI*, 2025.
- [6] Shihan Dou, Ming Zhang, Zhangyue Yin, Chenhao Huang, Yujiong Shen, Junzhe Wang, Jiayi Chen, Yuchen Ni, Junjie Ye, Cheng Zhang, et al. Cl-bench: A benchmark for context learning. *arXiv preprint arXiv:2602.03587*, 2026.
- [7] Mingxuan Du, Benfeng Xu, Chiwei Zhu, Xiaorui Wang, and Zhendong Mao. Deepresearch bench: A comprehensive benchmark for deep research agents, 2025. URL <https://arxiv.org/abs/2506.11763>.
- [8] Yiming Du, Bingbing Wang, Yang He, Bin Liang, Baojun Wang, Zhongyang Li, Lin Gui, Jeff Z. Pan, Ruifeng Xu, and Kam-Fai Wong. Memguide: Intent-driven memory selection for goal-oriented multi-session llm agents, 2025. URL <https://arxiv.org/abs/2505.20231>.
- [9] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitanaky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [10] Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, Zhaochun Ren, Nikos Aletras, Xi Wang, Han Zhou, and Zaiqiao Meng. A comprehensive survey of self-evolving ai agents: A new paradigm bridging foundation models and lifelong agentic systems, 2025. URL <https://arxiv.org/abs/2508.07407>.
- [11] Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory, 2026. URL <https://arxiv.org/abs/2508.06433>.
- [12] Ali Essam Ghareeb, Benjamin Chang, Ludovico Mitchener, Angela Yiu, Caralyn J. Szołkiewicz, Jon M. Laurent, Muhammed T. Razzak, Andrew D. White, Michaela M. Hinks, and Samuel G. Rodrigues. Robin: A multi-agent system for automating scientific discovery, 2025. URL <https://arxiv.org/abs/2505.13400>.
- [13] Google DeepMind. Gemini 3 flash: Frontier intelligence built for speed. <https://blog.google/products-and-platforms/products/gemini-3-flash/>, 2025.
- [14] Dongge Han, Camille Couturier, Daniel Madrigal Diaz, Xuchao Zhang, Victor Rühle, and Saravan Rajmohan. Legomem: Modular procedural memory for multi-agent llm systems for workflow automation, 2025. URL <https://arxiv.org/abs/2510.04851>.
- [15] Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, et al. Memoryarena: Benchmarking agent memory in interdependent multi-session agentic tasks. *ICML*, 2026.

- [16] Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in llm agents via incremental multi-turn interactions. *arXiv preprint arXiv:2507.05257*, 2025.
- [17] Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in llm agents via incremental multi-turn interactions. In *ICLR*, 2026.
- [18] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. Memory os of ai agent. In *EMNLP*, pages 25972–25981, 2025.
- [19] Sijia Li, Yuchen Huang, Zifan Liu, Zijian Li, Jingjing fu, Lei Song, Jiang Bian, Jun Zhang, and Rui Wang. Experience-evolving multi-turn tool-use agent with hybrid episodic-procedural memory, 2026. URL <https://arxiv.org/abs/2512.07287>.
- [20] Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu Wang, Jiawei Yang, Chen Tang, et al. Memos: A memory os for ai system. *arXiv preprint arXiv:2507.03724*, 2025.
- [21] Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.
- [22] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. *arXiv preprint arXiv:2402.17753*, 2024.
- [23] Andy Nguyen, Danh Doan, Hoang Pham, Bao Ha, Dat Pham, Linh Nguyen, Hieu Nguyen, Thien Nguyen, Cuong Do, Phat Nguyen, and Toan Nguyen. Byterover: Agent-native memory through llm-curated hierarchical context, 2026. URL <https://arxiv.org/abs/2604.01599>.
- [24] OpenAI. Introducing gpt-5. <https://openai.com/index/introducing-gpt-5/>, 2025.
- [25] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T Le, Samira Daruki, Xiangru Tang, et al. Reasoningbank: Scaling agent self-evolving with reasoning memory. In *ICLR*, 2026.
- [26] Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *ICML*, pages 48371–48392, 2025.
- [27] Hongjin Qian, Zhao Cao, and Zheng Liu. Memobrain: Executive memory as an agentic brain for reasoning. In *ACL Findings*, 2026.
- [28] Stephen Robertson and Hugo Zaragoza. *The probabilistic relevance framework: BM25 and beyond*, volume 4. Now Publishers Inc, 2009.
- [29] Rana Salama, Jason Cai, Michelle Yuan, Anna Currey, Monica Sunkara, Yi Zhang, and Yassine Benajiba. Meminsight: Autonomous memory augmentation for llm agents, 2025. URL <https://arxiv.org/abs/2503.21760>.
- [30] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.
- [31] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. In *ICLR*, 2021.
- [32] Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, et al. Agent kb: Leveraging cross-domain experience for agentic problem solving. *arXiv preprint arXiv:2507.06229*, 2025.
- [33] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In *ICML*, pages 63897–63911, 2025.

- [34] Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, Chi Wang, Shuo Chen, Fernando Pereira, Wang-Cheng Kang, and Derek Zhiyuan Cheng. Evo-memory: Benchmarking llm agent test-time learning with self-evolving memory. *arXiv preprint arXiv:2511.20857*, 2025.
- [35] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Long-memeval: Benchmarking chat assistants on long-term interactive memory. In *ICLR*, 2025.
- [36] Jialong Wu, Wenbiao Yin, Yong Jiang, Zhenglin Wang, Zekun Xi, Runnan Fang, Linhai Zhang, Yulan He, Deyu Zhou, Pengjun Xie, et al. Webwalker: Benchmarking llms in web traversal. In *ACL*, pages 10290–10305, 2025.
- [37] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. In *NeurIPS*, 2025.
- [38] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.
- [39] Hongli Yu, Tinghong Chen, Jiangtao Feng, Jiangjie Chen, Weinan Dai, Qiyang Yu, Ya-Qin Zhang, Wei-Ying Ma, Jingjing Liu, Mingxuan Wang, et al. Memagent: Reshaping long-context llm with multi-conv rl-based memory agent. *arXiv preprint arXiv:2507.02259*, 2025.
- [40] Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. Memevolve: Meta-evolution of agent memory systems. *arXiv preprint arXiv:2512.18746*, 2025.
- [41] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. Agentic context engineering: Evolving contexts for self-improving language models. In *ICLR*, 2026.
- [42] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, et al. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025.
- [43] Boyuan Zheng, Michael Y Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, et al. Skillweaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025.
- [44] Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, and Jun Wang. Memento: Fine-tuning llm agents without fine-tuning llms, 2025. URL <https://arxiv.org/abs/2508.16153>.

A Additional Details on EvoMemBench

A.1 Details of Datasets

All datasets used in our study are previously published benchmarks covering diverse domains including context learning, tool-use evaluation, memory-intensive agent interaction, profession-aligned real-world tasks, web navigation, and embodied decision-making environments. The detailed descriptions of these datasets and their associated evaluation metrics are as follows:

- **CL-Bench.** The CL-Bench [6] dataset is a comprehensive benchmark designed to evaluate language models’ context learning capability, i.e., the ability to acquire and apply new knowledge provided within the context rather than relying solely on pre-trained knowledge. It consists of 500 complex contexts, 1,899 tasks, and more than 31K verification rubrics spanning 4 major categories and 18 subcategories, including domain knowledge reasoning, procedural task execution, and empirical discovery. Unlike traditional long-context benchmarks that primarily focus on retrieval or reading comprehension, CL-Bench requires models to genuinely learn novel knowledge and rules from context. Model performance is evaluated using task-solving accuracy. In our experiments, CL-Bench is used to evaluate long-context memory retention and context-dependent reasoning ability.
- **BFCL-V3-MultiTurn.** The BFCL benchmark [26] is a large-scale evaluation suite for assessing tool-use and function-calling capabilities of large language models. The BFCL-V3-MultiTurn setting extends earlier versions by introducing multi-turn interactions, multi-step tool invocation, and parameter completion scenarios, enabling more realistic evaluation of agentic behavior in complex environments. The benchmark evaluates whether models can correctly select APIs, generate executable arguments, and maintain interaction consistency across multiple turns. Performance is primarily measured using function-calling accuracy and execution success rate.
- **MemoryAgentBench.** The MemoryAgentBench [16] dataset is a benchmark specifically designed for evaluating memory capabilities in LLM-based agents through incremental multi-turn interactions. The benchmark identifies four core competencies essential for memory agents, including accurate retrieval, test-time learning, long-range understanding, and selective forgetting. To simulate realistic conversational settings, the benchmark reformulates existing long-context datasets into incremental multi-turn interactions and introduces newly constructed tasks such as EventQA and FactConsolidation. Model performance is evaluated using memory retrieval accuracy and downstream task completion accuracy under progressively increasing interaction lengths.
- **xBench.** The xBench [4] benchmark is a profession-aligned real-world evaluation suite designed to measure productivity scaling behavior in autonomous agents. It contains realistic long-horizon tasks derived from professional workflows, including planning, coding, document processing, information analysis, and web interaction. Compared with conventional synthetic benchmarks, xBench emphasizes realistic task execution and environment interaction under practical constraints. Model performance is evaluated based on task success rate and execution quality across diverse professional domains.
- **WebWalkerQA.** The WebWalkerQA benchmark [36] is a web traversal and multi-hop web reasoning benchmark designed to evaluate LLM agents’ capability in navigating real-world websites and extracting information across multiple webpages. The associated WebWalkerQA task requires agents to sequentially browse webpages, collect evidence from different sources, and answer complex multi-hop questions. The benchmark evaluates both navigation success and final question-answering accuracy, making it suitable for assessing long-horizon planning, memory retention, and web interaction capabilities.
- **ALFWorld.** The ALFWorld [31] benchmark is a text-based embodied environment that aligns textual interaction with embodied household environments. Built upon the ALFRED embodied instruction-following benchmark, ALFWorld converts embodied tasks into text-based interactive environments involving navigation, object manipulation, and sequential decision-making in partially observable settings. Tasks typically require long-horizon planning and persistent memory of previously observed states. Model performance is evaluated using task success rate and goal completion accuracy.

A.2 Details of Metrics

We evaluate memory methods from two complementary perspectives: task performance and efficiency. Different metrics are used depending on the type of evaluation setting.

Answer Accuracy. For knowledge-oriented settings, including INEP-KNOW and CROSSEP-KNOW, we report *answer accuracy*, defined as the proportion of evaluation instances answered correctly:

$$\text{Acc} = \frac{1}{N} \sum_{i=1}^N \text{Correct}(i),$$

where $\text{Correct}(i) \in \{0, 1\}$ indicates whether the prediction for instance i is correct.

Success Rate. For execution-oriented settings, including INEP-EXEC, CROSSEP-TOOL, CROSSEP-WEB, and CROSSEP-EMB, we report *success rate*. Let $\text{Success}(i) \in \{0, 1\}$ indicate whether the agent successfully completes the final goal of task i according to the task-specific evaluator. The success rate is defined as

$$\text{SR} = \frac{1}{N} \sum_{i=1}^N \text{Success}(i).$$

This metric captures complete task accomplishment and is therefore a strict measure of end-to-end execution quality.

Progress Score. In INEP-EXEC and CROSSEP-TOOL, we additionally report *progress score* to characterize partial task completion. These tasks involve multi-turn tool-use trajectories in which an agent may correctly complete earlier turns or intermediate requirements even if it eventually fails to solve the full task. For task i , let T_i denote the number of evaluated turns or execution checkpoints, and let $\text{Pass}(i, t) \in \{0, 1\}$ indicate whether the agent satisfies the evaluator at checkpoint t . The task-level progress score is

$$\text{PS}_i = \frac{1}{T_i} \sum_{t=1}^{T_i} \text{Pass}(i, t),$$

and the overall progress score is

$$\text{PS} = \frac{1}{N} \sum_{i=1}^N \text{PS}_i.$$

Unlike success rate, which only rewards fully completed tasks, progress score provides a finer-grained view of how far an agent proceeds during execution.

Average Steps. We report *average steps* for INEP-EXEC, CROSSEP-TOOL, CROSSEP-WEB, and CROSSEP-EMB. For each task i , let $\text{Steps}(i)$ denote the number of agent execution steps taken during the trajectory, where a step follows the native interaction unit of the corresponding environment or agent framework. Average steps is computed as

$$\text{AS} = \frac{1}{N} \sum_{i=1}^N \text{Steps}(i).$$

This metric reflects interaction efficiency: lower values indicate that the agent reaches termination with fewer execution steps. We report it together with success rate to distinguish methods that are merely more successful from those that are both successful and efficient.

Token Usage. We use *token usage* as a general efficiency metric. For each evaluation instance, token usage includes all LLM inference tokens consumed by both the task agent and the memory module, covering input and output tokens across all model calls. Let $\mathcal{C}_i$ denote the set of LLM calls made for instance i , and let $\text{Tok}^{\text{in}}(c)$ and $\text{Tok}^{\text{out}}(c)$ denote the input and output tokens of call c . The token usage for instance i is

$$\text{TU}_i = \sum_{c \in \mathcal{C}_i} \left(\text{Tok}^{\text{in}}(c) + \text{Tok}^{\text{out}}(c) \right).$$

We report token usage to quantify the computational overhead introduced by different memory mechanisms.

A.3 Details of Baselines

We compare our framework against representative baselines spanning six categories, including long-context LLMs, retrieval-augmented memory systems, short-term memory architectures, general long-term memory systems, procedural long-term memory methods, and meta-evolution memory frameworks. Below we provide detailed descriptions of these baselines.

- **Long-Context LLMs.**

- **Gemini-3-Flash** [13] is a proprietary long-context language model developed by Google. It supports efficient processing of extremely long contexts and demonstrates strong performance on retrieval, reasoning, and multimodal understanding tasks. Its optimized sparse attention and long-context inference mechanisms make it a strong baseline for large-scale memory-intensive applications. Its efficient long-context decoding strategy also enables stable performance under extremely large context windows with relatively low inference latency.
- **GPT-5-mini** [24] is a lightweight variant of OpenAI’s GPT-5 family optimized for efficient inference and long-context interaction. Despite its compact size, it maintains strong instruction-following and reasoning abilities, serving as a representative efficient long-context proprietary model. The model is particularly suitable for interactive agent settings requiring fast response generation under constrained computational budgets.
- **DeepSeek-V3.2** [21] is an advanced mixture-of-experts (MoE) language model developed by DeepSeek AI. It incorporates efficient routing mechanisms and large-scale continual pretraining to support long-context reasoning, coding, and retrieval-intensive tasks with strong scalability. Its MoE architecture allows scalable activation of specialized experts, improving efficiency for heterogeneous reasoning and retrieval workloads.

- **Retrieval-Augmented Memory.**

- **BM25** [28] is a classical sparse retrieval algorithm based on term-frequency and inverse document frequency statistics. It remains a strong lexical retrieval baseline for long-context question answering and memory retrieval tasks. Due to its exact lexical matching behavior, BM25 is particularly robust for retrieving sparse factual memories and keyword-centric information.
- **Qwen3-Emb-4B** [42] is a 4B parameter embedding model built upon the Qwen3 foundation model. It adopts instruction-aware embedding training and supports multilingual semantic retrieval, providing strong dense retrieval capability for memory-intensive agent systems. Its strong semantic embedding capability enables effective retrieval even when queries and memories exhibit substantial lexical variation.
- **GraphRAG** [9] is a graph-based retrieval-augmented generation framework that first constructs an entity-centric knowledge graph from source documents, then performs hierarchical community summarization for query answering. Compared with conventional vector RAG systems, GraphRAG is particularly effective for corpus-level reasoning and global summarization tasks. The graph-based organization additionally improves multi-hop retrieval and relation-aware reasoning across large document collections.

- **Short-Term Memory.**

- **MemAgent** [39] reformulates long-context modeling as a reinforcement learning problem with a fixed-size memory buffer. The model iteratively reads document chunks and updates compressed memory representations through overwrite operations. It employs Multi-Conversation DAPO training and achieves strong extrapolation ability under linear computational complexity. The overwrite-based memory update mechanism enables bounded memory usage while maintaining long-context reasoning capability.
- **MemoBrain** [27] is a short-term memory framework designed for dynamic conversational reasoning. It continuously compresses recent interaction histories into compact memory states, enabling efficient context maintenance and rapid memory updates during agent interaction. Its executive-style memory abstraction mechanism is particularly designed to support dynamic reasoning over evolving conversational contexts.

- **General Long-Term Memory.**

- **Mem0** [5] is a scalable long-term memory architecture for conversational agents. It incrementally extracts salient information from interactions and manages memory using four

operations: ADD, UPDATE, DELETE, and NOOP. The framework also introduces graph-based memory representations for enhanced relational reasoning. Its fine-grained memory operation design allows flexible incremental memory maintenance across long-term interactions.

- **A-MEM** [37] is a structured memory framework that organizes historical interactions into interconnected semantic notes. It emphasizes adaptive memory organization and long-term retrieval for persistent agent reasoning across sessions. The framework further supports memory association and semantic linking across temporally distant interactions.
- **MemOS** [20] is an operating-system-inspired memory management framework for AI agents. It adopts hierarchical storage and segmented memory management strategies to coordinate memory allocation, retrieval, and long-term persistence across complex interactions. Its operating-system-inspired abstraction enables modular coordination between memory scheduling, retrieval, and persistence modules.
- **MemoryOS** [18] introduces a three-tier memory hierarchy consisting of short-term memory (STM), mid-term memory (MTM), and long-term personal memory (LPM). The framework integrates memory storage, updating, retrieval, and response generation modules, enabling dynamic conversational memory evolution and long-term personalization. The hierarchical memory design enables adaptive retention of both recent interaction context and long-term personalized knowledge.

• **Procedural Long-Term Memory.**

- **AWM (Agent Workflow Memory)** [33] enables agents to induce reusable workflows from historical trajectories and store them as structured memory units. During inference, the agent retrieves relevant workflows to guide future planning and execution. AWM supports both offline workflow induction from demonstrations and online induction from self-generated successful trajectories. By reusing previously induced workflows, AWM improves planning efficiency and reduces redundant exploration during agent execution.
- **SkillWeaver** [43] is a self-improving web-agent framework that autonomously discovers, synthesizes, and refines reusable skills represented as executable APIs. It consists of three stages: skill proposal, skill synthesis, and skill honing, enabling continuous procedural skill accumulation through environment interaction. The framework enables continual procedural self-improvement through autonomous skill refinement across repeated web interactions.
- **AgentKB** [32] is a cross-framework knowledge-sharing infrastructure for heterogeneous agent systems. It abstracts execution trajectories into reusable experience units and employs hybrid retrieval together with disagreement-gated integration to enable stable cross-agent knowledge transfer.
- **ACE** [41] is a procedural memory framework that accumulates executable agent experiences as reusable action templates. It focuses on experience abstraction and adaptive execution reuse for complex sequential decision-making tasks. The framework is particularly effective for repeated sequential tasks that benefit from reusable execution patterns.
- **ReasoningBank** [25] is a reasoning-oriented memory repository that stores structured reasoning traces and intermediate decision trajectories. It enhances long-horizon planning by retrieving relevant reasoning patterns from historical problem-solving experiences. The stored reasoning trajectories additionally support retrieval-based reasoning generalization across related problem-solving tasks.

• **Meta-Evolution Memory.**

- **MemEvolve** [40] is a meta-evolutionary memory framework that continuously refines and evolves stored memories through iterative self-optimization. The framework dynamically updates memory structures based on long-term task feedback, enabling adaptive memory evolution and continual learning. Its self-evolution mechanism enables continuous adaptation of memory structures under changing task distributions and environments.

Notably, since different settings expose different memory interfaces, we evaluate only applicable methods in each setting: procedural long-term memory is excluded from in-episode knowledge evolution, and short-term memory is excluded from cross-episode settings.

B Additional Details on Implementations

B.1 Prompt for INEP-EXEC and CROSS-TOOL Construction

You are a dataset constructor for a multi-session agent-memory benchmark.

Your task is to jointly rewrite a multi-turn tool-use example by splitting any turn whose ground-truth contains multiple consecutive actions into multiple turns.

Goal:

Transform the original example into a more explicit multi-session sequence of dependent subtasks, while preserving the original semantics and increasing the need for cross-turn memory.

Core principle:

After splitting, if a later rewritten turn depends on an object, result, or state established in an earlier rewritten turn, that dependency should be expressed implicitly whenever possible, rather than by explicitly restating the exact entity name, ID, parameter value, or resolved result.

Rules:

- Only split a turn if its ground-truth contains multiple actions that form a meaningful sequential dependency.
- Split queries and split ground-truth must remain semantically aligned.
- The new ground-truth must be derivable from the old ground-truth by ordered slicing only.
- Do NOT invent new actions.
- Do NOT delete actions.
- Do NOT reorder actions.
- Do NOT add new constraints.
- Do NOT change action arguments unless the meaning stays exactly the same.

A turn is splittable only if:

1. Its ground-truth contains 2 or more actions; and
2. those actions form a natural user-level dependency, such as:
 - lookup -> use result
 - authenticate -> perform protected action
 - prepare state -> execute action
 - retrieve current info -> commit operation

Do NOT split:

- single-action turns
- turns whose actions are merely parallel
- turns whose internal steps are only low-level implementation details without meaningful subtask boundaries

Rewrite guidance:

- Each split query must sound natural as a user request.
- The first split query should request the first meaningful substep.
- Later split queries should naturally depend on earlier ones.
- Prefer implicit cross-turn references over explicit restatement.
- Preserve the original user intent and style.
- Keep the whole rewritten dialogue coherent.

Important implicit-reference rule:

When a later rewritten turn depends on something already established in an earlier rewritten turn, do NOT explicitly restate that earlier-established object if the dependency can be expressed naturally and unambiguously through an implicit reference.

Prefer references like:

- "the directory you created"
- "the one we just found"
- "that order"
- "those details"
- "the trip"
- "the message I sent"
- "the stock we just checked"
- "the reservation"
- "the result"

Avoid explicit restatement like:

- exact directory/file names already introduced in an earlier split turn
- exact IDs already resolved in an earlier split turn
- exact parameter values already determined in an earlier split turn
- exact entity names that should now be recoverable only from memory

However, do NOT make the query ambiguous or underspecified.

If an explicit mention is necessary to preserve clarity or avoid multiple valid interpretations, it may be kept.

What to keep explicit:

- Information newly introduced in the current rewritten turn
- Constraints that first appear in the current rewritten turn
- Information necessary to avoid ambiguity

What to make implicit when possible:

- Objects created earlier
- Results computed earlier
- Items selected earlier from a list
- State changes caused earlier
- Previously resolved destinations, bookings, tickets, orders, directories, messages, or chosen entities

Ground-truth handling:

- For a split turn, partition the original action list into consecutive slices.
- Each new query turn must align to exactly one consecutive slice.
- Concatenating all new slices for that original turn must exactly recover the original action list.

Hard constraints:

1. Never split a single-action turn.
2. Never merge two original turns.
3. Never split a turn into more parts than the number of actions in its original ground-truth.
4. Every split part must contain at least one action.
5. Later split queries may refer to earlier split queries, but must not depend on future turns.
6. Keep unsplit turns semantically unchanged except for light editing if needed for flow.

7. After splitting, do not expose cross-turn dependencies more explicitly than in the original task.
8. Prefer memory-dependent phrasing over answer-revealing phrasing.

Positive example:

Original turn:

Query: Move 'final_report.pdf' within document directory to 'temp' directory in document. Make sure to create the directory

Ground truth:

```
- cd(folder='document')  
- mkdir(dir_name='temp')  
- mv(source='final_report.pdf', destination='temp')
```

Good rewrite:

Turn 1

Query: Change to the document directory and create a 'temp' directory there.

Ground truth:

```
- cd(folder='document')  
- mkdir(dir_name='temp')
```

Turn 2

Query: With that ready, move 'final_report.pdf' into the directory you created.

Ground truth:

```
- mv(source='final_report.pdf', destination='temp')
```

Negative rewrite:

Turn 1

Query: Change to the document directory and create a 'temp' directory there.

Ground truth:

```
- cd(folder='document')  
- mkdir(dir_name='temp')
```

Turn 2

Query: With that ready, move 'final_report.pdf' into the 'temp' directory within document.

Ground truth:

```
- mv(source='final_report.pdf', destination='temp')
```

Why negative:

Because the second turn explicitly restates the key object ('temp' directory) that should instead be recoverable from memory.

Return JSON only in the format:

```

"id": "<same id as input>",
"rewritten_question": [
  {"role": "user", "content": "..."},
  ...
],
"rewritten_ground_truth": [
  "action1(...)",
  ...
],
"split_map": [
  "original_turn": 1,
  "was_split": true,
  "new_turns": [1, 2]
]

```

Before answering, internally verify:

1. Every original action appears exactly once in rewritten_ground_truth.
2. The global action order is preserved.
3. Each rewritten query matches its rewritten ground-truth.
4. No new actions were introduced.
5. No original actions were dropped.
6. The rewritten dialogue is coherent.
7. Later rewritten turns use implicit references wherever appropriate.
8. Cross-turn dependencies are not unnecessarily exposed by explicit restatement.

Now process the following input.

Query JSON:

<PASTE_QUERY_JSON_HERE>

Ground Truth JSON:

<PASTE_GROUND_TRUTH_JSON_HERE>

B.2 Details of Experiments.

To ensure a unified implementation, each memory method is wrapped with two interfaces: `utilize` and `update`. The `utilize` interface takes the current query or state as input and returns task-relevant memory, such as retrieved entries, procedural guidance, or a compact working context. The returned memory is then used as part of the agent prompt. The `update` interface incorporates newly observed information into the memory. The update schedule depends on the evaluation setting. In in-episode settings, memory is initialized at the beginning of each episode, updated during the episode, and cleared after the episode ends. For InEp-Know, memory is updated after each progressive information block. For InEp-Exec, memory is updated after each interaction turn. In cross-episode settings, memory is updated after each episode using the episode record or full trajectory, and is reused in later episodes from the same context or environment. Memory is reset when moving to a new independent context or environment group. For cross-environment execution transfer, memory is first built on the source environment and then frozen during evaluation on the target environment. For InEp-Exec, we evaluate context budgets of 16K, 32K, 64K, and 128K. When the prompt exceeds the budget, we keep the current task input and memory, and truncate early history. Unless a method requires its own embedding model, we use `text-embedding-v4` from DashScope as the default embedding backend. We set the retrieval top- k to 10 for knowledge-oriented tasks and 3 for execution-oriented tasks.

C Additional Experimental Results

This section provides supplementary results for INEP-EXEC, cross-environment transfer in cross-episode execution settings, and efficiency analysis in cross-episode settings.

C.1 Additional Results for Cross-Environment Transfer in Cross-Episode Execution Settings

We provide the full cross-environment transfer visualizations for CROSSEP-EMB and CROSSEP-TOOL.

C.1.1 CROSSEP-EMB

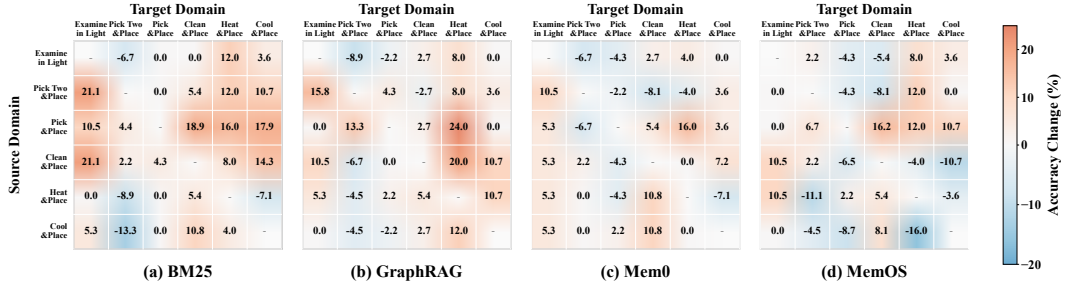


Figure 5: Cross-environment transfer results in CROSSEP-EMB for BM25, GraphRAG, Mem0, and MemOS.

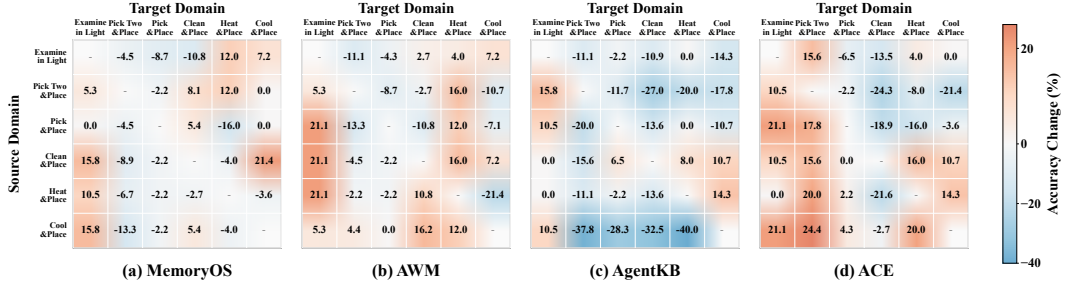


Figure 6: Cross-environment transfer results in CROSSEP-EMB for MemoryOS, AWM, AgentKB, and ACE.

C.1.2 CROSSEP-TOOL

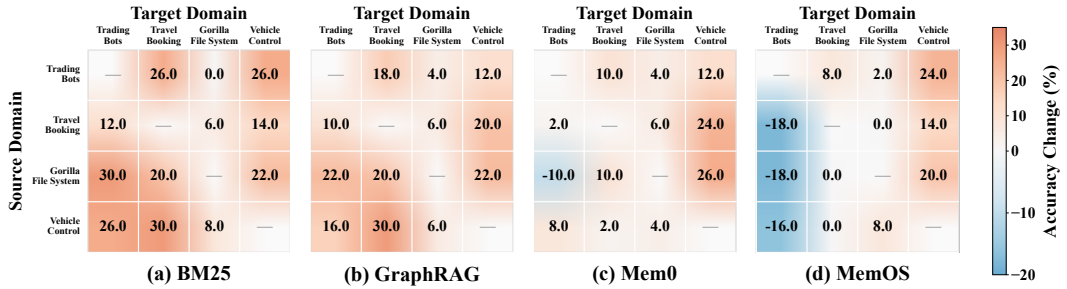


Figure 7: Cross-environment transfer results in CROSSEP-TOOL for BM25, GraphRAG, Mem0, and MemOS.

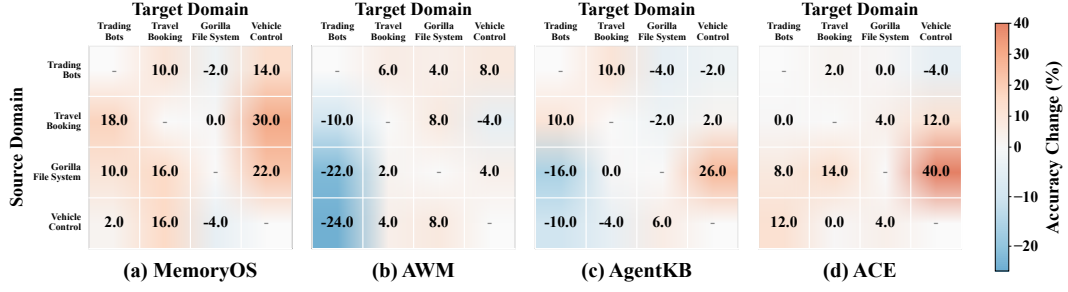


Figure 8: Cross-environment transfer results in CROSSEP-TOOL for MemoryOS, AWM, AgentKB, and ACE.

C.2 Additional Efficiency Results for Cross-Episode Settings

We report token usage for cross-episode knowledge evolution and both average steps and token usage for cross-episode execution settings.

Table 7: Efficiency comparison on CROSSEP-KNOW, measured by token usage. Lower is better. The best result in each column is bolded and the runner-up is underlined.

Method	Domain Knowledge	Empirical Discovery	Procedural Task	Rule System Application
Gemini-3-Flash	15465	18546	14701	14942
GPT-5-mini	17188	20278	16636	15295
DeepSeek-V3.2	6416	8145	6021	5613
BM25	14610	16224	14351	14113
Qwen3-Emb-4B	14576	16655	14609	14056
GraphRAG	15850	17993	15773	15386
Mem0	22852	26336	21932	21320
A-MEM	31939	35668	31854	31207
MemOS	<u>7640</u>	<u>9456</u>	<u>7180</u>	6910
MemoryOS	18506	23676	20074	19409
AWM	14604	18092	13776	13054
SkillWeaver	14407	18028	13708	12683
AgentKB	10002	11793	9532	9241
ACE	20802	23612	20436	19495
ReasoningBank	13632	15440	13329	13278
MemEvolve	9876	11734	9381	9080

Table 8: Efficiency comparison on CROSSEP-EMB tasks measured by average steps. Lower is better. The best result in each column is bolded and the runner-up is underlined.

Method	Examine in Light	Pick & Place	Clean & Place	Cool & Place	Heat & Place	Pick Two & Place	Avg. (Weighted)
Gemini-3-Flash	15.00	13.20	16.55	15.43	15.60	18.76	15.85
GPT-5-mini	14.74	11.85	18.70	18.11	16.16	18.36	16.27
DeepSeek-V3.2	11.16	9.22	15.30	15.71	16.32	17.16	14.11
BM25	11.26	8.89	14.49	14.68	14.08	15.51	13.10
Qwen3-Emb-4B	12.37	8.57	13.27	14.68	14.84	14.84	12.89
GraphRAG	12.63	<u>8.52</u>	13.32	14.46	13.44	14.42	12.57
AgentKB	12.21	11.37	12.05	18.64	12.16	13.76	13.23
ACE	12.79	8.98	15.76	10.68	11.40	14.56	12.39
ReasoningBank	10.11	10.13	12.00	13.39	14.44	14.31	12.41
Mem0	11.05	9.41	13.50	14.23	14.33	16.78	13.27
AWM	<u>9.58</u>	9.98	12.19	<u>12.32</u>	11.92	15.82	<u>12.23</u>
SkillWeaver	12.84	10.37	15.00	15.50	13.48	16.04	13.85
MemEvolve	12.89	10.00	14.35	13.79	12.04	<u>13.53</u>	12.66
MemOS	12.53	9.96	13.68	14.89	15.36	17.02	13.85
MemoryOS	11.00	8.61	13.32	14.29	14.80	16.22	12.99
A-MEM	9.37	8.89	<u>12.08</u>	14.14	<u>11.60</u>	12.58	11.43

Table 9: Efficiency comparison on CROSSEP-EMB tasks measured by token usage. Lower is better. The best result in each column is bolded and the runner-up is underlined.

Method	Examine in Light	Pick & Place	Clean & Place	Cool & Place	Heat & Place	Pick Two & Place	Avg. (Weighted)
GPT-5-mini	40382	<u>26094</u>	63944	67886	56080	43831	48043
Gemini-3-Flash	30384	26783	61690	61724	51490	40447	44637
DeepSeek-V3.2	25653	16001	46105	51950	53153	<u>38274</u>	37175
BM25	31771	65227	118241	103920	80766	126938	93100
Qwen3-Emb-4B	32634	52669	82326	86253	78910	105429	76105
GraphRAG	37746	57691	100512	92602	83649	108669	83320
AgentKB	37423	33358	45436	75351	<u>48484</u>	39087	45037
ACE	60520	43583	82866	59019	57865	58046	59660
ReasoningBank	33244	29045	45198	57985	61394	42740	43609
Mem0	40418	31188	50676	59465	59930	53237	48182
AWM	<u>28796</u>	31693	51074	58498	55304	50474	45933
SkillWeaver	45981	34945	62774	71643	62964	49480	53052
MemEvolve	38887	26915	49619	<u>52045</u>	41926	35635	<u>39609</u>
MemOS	39318	26133	51165	65274	60703	49216	47011
MemoryOS	77670	55638	112278	134858	136843	102816	100066
A-MEM	53004	37768	67852	73298	60338	58932	57338

Table 10: Efficiency comparison on CROSSEP-TOOL, measured by average steps. Lower is better. The best result in each column is bolded and the runner-up is underlined.

Method	Gorilla File System	Vehicle Control	Trading Bots	Travel Booking	Average
Gemini-3-Flash	<u>12.40</u>	14.08	12.28	12.86	12.90
GPT-5-mini	12.18	12.70	<u>10.40</u>	10.68	11.49
DeepSeek-V3.2	14.82	<u>13.44</u>	<u>10.40</u>	10.90	<u>12.39</u>
BM25	18.08	15.94	12.52	11.80	14.59
Qwen3-Emb-4B	17.50	15.54	12.16	10.82	14.01
GraphRAG	19.20	14.28	11.78	11.00	14.07
ReasoningBank	18.92	15.66	12.16	12.16	14.73
ACE	18.62	16.02	7.94	<u>10.60</u>	13.30
Mem0	15.00	14.94	10.86	11.42	13.06
AgentKB	23.44	20.16	13.26	12.50	17.33
AWM	17.70	14.48	12.00	10.42	13.65
SkillWeaver	16.88	13.96	10.88	11.36	13.27
MemEvolve	17.90	15.54	11.18	11.32	13.99
MemOS	15.76	14.30	10.32	11.40	12.95
MemoryOS	19.38	15.02	15.82	12.30	15.63
A-MEM	19.48	13.98	12.56	10.76	13.20

Table 11: Efficiency comparison on CROSSEP-TOOL, measured by token usage. Lower is better. The best result in each column is bolded and the runner-up is underlined.

Method	Gorilla File System	Vehicle Control	Trading Bots	Travel Booking	Average
Gemini-3-Flash	<u>46221</u>	<u>51252</u>	<u>37416</u>	57043	47983
GPT-5-mini	36699	39325	26520	37108	34913
DeepSeek-V3.2	67664	60192	39266	<u>56026</u>	55787
BM25	254111	221727	167244	168780	202966
Qwen3-Emb-4B	230714	203511	167175	144507	186477
GraphRAG	270176	193998	158727	154069	194243
ReasoningBank	102835	83237	59223	74413	79927
ACE	139608	127034	67865	102581	109272
Mem0	79390	79831	54195	70791	71052
AgentKB	145361	134314	78242	91854	112333
AWM	92055	74334	58037	60606	71258
SkillWeaver	83388	67458	47794	63644	65571
MemEvolve	97691	85105	54285	71952	77258
MemOS	79324	70373	44716	65292	64926
MemoryOS	125445	102908	181489	102188	128033
A-MEM	217512	153201	210504	143973	181298

Table 12: Efficiency comparison on CROSSEP-WEB, measured by average steps. Lower is better. The best result in each column is bolded and the runner-up is underlined.

Method	xbench	WebWalkerQA	xbench→WebWalkerQA	WebWalkerQA→xbench
Gemini-3-Flash	11.10	9.95	-	-
GPT-5-mini	12.63	10.07	-	-
DeepSeek-V3.2	10.01	8.12	-	-
BM25	<u>10.92</u>	10.24	10.44	12.06
Qwen3-Emb-4B	12.41	9.82	9.84	11.65
GraphRAG	11.68	<u>8.81</u>	10.55	11.40
ReasoningBank	11.92	9.59	9.29	11.39
ACE	12.92	10.09	10.48	10.59
Mem0	12.20	9.74	10.01	12.79
AgentKB	12.62	9.85	10.46	11.49
AWM	12.19	9.51	10.09	<u>10.94</u>
SkillWeaver	12.08	9.62	9.96	11.79
MemEvolve	12.31	9.84	<u>9.81</u>	11.65
MemOS	11.17	9.84	10.17	14.49
MemoryOS	14.24	12.08	12.55	13.89
A-MEM	14.20	11.31	12.35	12.65

Table 13: Efficiency comparison on CROSS-EP-WEB, measured by token usage. Lower is better. The best result in each column is bolded and the runner-up is underlined.

Method	xbench	WebWalkerQA	xbench→WebWalkerQA	WebWalkerQA→xbench
Gemini-3-Flash	<u>269911</u>	140189	-	-
GPT-5-mini	<u>306538</u>	138425	-	-
DeepSeek-V3.2	377624	164081	-	-
BM25	384243	180381	302560	431878
Qwen3-Emb-4B	451976	262780	256687	412250
GraphRAG	432288	220649	289542	402795
ReasoningBank	287683	<u>132898</u>	131799	281394
ACE	481140	366907	409670	815386
Mem0	316996	138051	<u>137435</u>	317055
AgentKB	344953	136622	156246	<u>266775</u>
AWM	301893	138651	150348	259937
SkillWeaver	293638	130577	143156	286286
MemEvolve	294272	142265	145114	284052
MemOS	260389	140872	156330	316401
MemoryOS	280934	161801	151790	309810
A-MEM	553566	340888	396823	454250

C.3 Additional Results for INEP-EXEC

We report complementary results for INEP-EXEC, including progress score and average steps under different context budgets.

Table 14: Progress score (%) comparison across different memory methods on INEP-EXEC under different context lengths. Unless otherwise specified, all memory agents use DeepSeek-V3.2 as the backbone. Best results are bolded, and runner-up results among memory methods are underlined.

Method	Gorilla File System				Vehicle Control				Trading Bots				Travel Booking				Overall			
	16K	32K	64K	128K	16K	32K	64K	128K	16K	32K	64K	128K	16K	32K	64K	128K	16K	32K	64K	128K
<i>Long-Context LLM</i>																				
Gemini-3-Flash	77.4	74.0	77.6	75.2	76.9	80.6	77.2	71.2	52.5	91.2	85.2	88.7	76.2	78.5	75.2	70.1	70.7	81.0	78.8	76.3
GPT-5-mini	58.7	61.8	63.7	62.4	62.3	59.9	56.2	55.9	48.5	78.7	82.0	80.6	56.7	53.2	57.7	54.0	56.5	63.4	64.9	63.2
DeepSeek-V3.2	76.9	75.5	73.2	76.2	57.2	60.1	60.2	60.5	29.4	50.3	54.6	52.2	40.2	37.9	35.8	40.3	50.9	56.0	55.9	57.3
<i>Retrieval-Augmented Memory</i>																				
BM25	77.4	<u>80.9</u>	72.8	76.5	<u>68.8</u>	63.6	58.3	65.0	34.1	63.2	54.5	48.7	47.6	47.4	38.8	45.0	57.0	<u>63.8</u>	56.1	58.8
Qwen3-Emb-4B	58.7	81.1	68.7	<u>80.3</u>	65.0	64.7	60.6	58.4	37.7	47.6	55.6	50.0	54.7	45.3	49.1	46.3	54.0	59.7	58.5	58.8
GraphRAG	76.9	80.2	80.2	79.1	61.5	58.8	64.4	60.6	33.8	43.4	56.8	47.5	45.0	48.4	50.8	45.4	54.3	57.7	63.0	58.2
<i>Short-Term Memory</i>																				
MemAgent	70.6	74.8	<u>79.8</u>	72.0	56.5	51.3	52.4	56.4	28.8	49.8	52.9	<u>60.0</u>	28.0	43.6	36.2	37.3	46.0	54.9	55.3	56.4
MemoBrain	69.0	72.5	78.6	73.4	60.7	54.5	60.3	61.9	24.8	44.9	57.0	50.4	42.1	35.4	40.7	44.9	49.1	51.8	59.1	57.6
<i>General Long-Term Memory</i>																				
Mem0	<u>78.7</u>	70.4	70.9	82.5	63.9	62.0	59.8	58.5	36.8	56.6	47.0	46.4	46.2	41.0	43.1	47.2	56.4	57.5	55.2	58.6
A-MEM	76.4	75.0	76.2	72.8	67.2	62.9	54.8	65.7	37.2	43.8	56.4	47.2	41.3	48.4	44.7	43.3	55.5	57.5	58.0	57.3
MemOS	76.9	75.2	79.0	76.0	63.6	55.4	61.0	57.2	32.8	51.9	55.2	53.9	45.7	34.4	37.0	35.3	54.8	54.2	58.0	55.6
MemoryOS	75.4	79.9	72.5	73.6	57.8	54.9	62.6	61.2	26.4	53.6	50.0	21.5	38.7	42.1	39.5	49.8	49.6	57.6	56.1	51.5
<i>Procedural Long-Term Memory</i>																				
AWM	75.2	74.9	78.3	77.9	60.0	65.8	63.8	50.2	37.0	70.7	85.0	52.3	43.9	<u>50.3</u>	47.0	42.4	54.0	65.4	68.5	55.7
SkillWeaver	75.1	72.6	73.8	74.9	61.3	63.0	<u>69.4</u>	67.4	31.4	49.6	67.7	52.3	46.4	47.0	<u>50.7</u>	<u>51.5</u>	53.5	58.0	65.4	61.5
AgentKB	75.0	79.9	77.2	74.0	22.6	61.6	58.1	74.7	46.2	48.3	<u>75.6</u>	66.6	44.7	39.1	36.1	39.9	47.1	57.2	61.7	63.8
ACE	76.6	70.8	78.4	78.1	59.3	51.8	53.7	59.9	26.8	48.0	52.9	52.6	45.3	40.9	43.7	41.6	52.0	52.8	57.2	58.0
ReasoningBank	83.3	75.4	74.6	79.0	69.6	<u>65.9</u>	64.7	67.5	32.3	50.9	45.0	56.7	50.4	60.8	45.2	53.6	<u>58.9</u>	63.2	57.4	<u>64.2</u>
<i>Meta-Evolution Memory</i>																				
MemEvolve	74.1	73.5	74.8	79.1	67.7	66.1	79.1	<u>69.0</u>	<u>41.0</u>	<u>64.2</u>	75.0	58.3	<u>53.9</u>	45.1	44.2	50.8	59.2	62.2	<u>68.3</u>	64.3

Table 15: Efficiency comparison on INEP-EXEC under the 16K setting, measured by average steps. Lower is better. The best result in each column is bolded and the runner-up is underlined.

Method	Gorilla File System	Vehicle Control	Trading Bots	Travel Booking	Average
Gemini-3-Flash	12.73	13.73	17.02	13.22	14.11
GPT-5-mini	12.56	<u>12.72</u>	17.06	10.16	13.12
DeepSeek-V3.2	15.58	13.92	10.12	10.74	<u>12.59</u>
BM25	16.40	14.64	12.28	11.00	13.58
Qwen3-Emb-4B	16.32	15.30	16.32	11.12	14.77
GraphRAG	16.46	14.40	11.24	11.00	13.28
MemAgent	15.48	14.20	<u>10.16</u>	<u>10.26</u>	12.53
MemoBrain	15.66	13.83	<u>12.35</u>	<u>10.88</u>	13.18
Mem0	16.40	14.10	12.32	11.06	12.99
A-MEM	15.18	14.02	12.08	10.58	12.96
MemOS	15.60	14.22	11.96	10.70	13.12
MemoryOS	16.14	14.62	11.36	10.48	13.15
AWM	20.40	14.52	12.90	10.98	14.70
SkillWeaver	16.88	14.02	10.42	11.00	13.08
AgentKB	30.58	12.32	18.20	12.32	18.36
ACE	17.20	14.82	12.72	11.92	14.16
ReasoningBank	19.76	15.56	13.86	12.70	15.47
MemEvolve	17.86	14.76	11.46	11.32	13.85

Table 16: Efficiency comparison on INEP-EXEC under the 16K setting, measured by token usage. Lower is better. The best result in each column is bolded and the runner-up is underlined.

Method	Gorilla File System	Vehicle Control	Trading Bots	Travel Booking	Average
Gemini-3-Flash	57428	86939	119487	77563	84594
GPT-5-mini	46265	66404	129870	46859	72349
DeepSeek-V3.2	82104	97262	<u>54125</u>	65028	<u>74630</u>
BM25	122644	123672	101003	89700	109255
Qwen3-Emb-4B	121698	127476	261780	91906	150715
GraphRAG	127673	125803	87847	97989	109828
MemAgent	82054	96510	45461	61729	71438
MemoBrain	126606	141363	155852	89061	127947
Mem0	133415	153265	116864	119538	130771
A-MEM	105199	116426	97830	92583	103009
MemOS	101334	121539	114826	83931	105407
MemoryOS	116687	119788	70983	85381	98210
AWM	140220	115285	81033	79209	103937
SkillWeaver	95428	97980	59924	71618	81237
AgentKB	227756	99861	115654	87945	132804
ACE	143073	165384	154381	128670	147877
ReasoningBank	153539	157398	176499	117945	151345
MemEvolve	106880	112956	73194	76621	92412

Table 17: Efficiency comparison on INEP-EXEC under the 32K setting, measured by average steps. Lower is better. The best result in each column is bolded and the runner-up is underlined.

Method	Gorilla File System	Vehicle Control	Trading Bots	Travel Booking	Average
Gemini-3-Flash	13.17	13.80	12.46	13.54	13.24
GPT-5-mini	12.40	12.80	10.63	10.52	11.59
DeepSeek-V3.2	15.60	<u>13.36</u>	11.36	10.04	12.59
BM25	16.56	14.40	11.28	11.04	13.32
Qwen3-Emb-4B	15.90	14.44	12.36	11.36	13.52
GraphRAG	15.62	14.46	11.60	11.34	13.26
MemAgent	15.62	14.10	10.56	10.80	12.77
MemoBrain	15.30	13.54	<u>10.32</u>	<u>10.28</u>	<u>12.36</u>
Mem0	16.14	14.20	10.78	10.84	12.99
A-MEM	16.66	14.40	10.78	10.70	13.13
MemOS	16.68	14.28	10.62	10.32	12.97
MemoryOS	16.00	14.26	11.68	11.20	13.29
AWM	22.52	14.66	12.90	11.18	15.32
SkillWeaver	16.62	14.44	10.28	10.58	12.98
AgentKB	27.72	17.66	17.28	11.58	18.56
ACE	18.58	14.96	11.70	11.64	14.22
ReasoningBank	20.38	15.76	11.34	12.46	14.98
MemEvolve	18.02	14.42	11.28	11.24	13.74

Table 18: Efficiency comparison on INEP-EXEC under the 32K setting, measured by token usage. Lower is better. The best result in each column is bolded and the runner-up is underlined.

Method	Gorilla File System	Vehicle Control	Trading Bots	Travel Booking	Average
Gemini-3-Flash	<u>59242</u>	<u>84186</u>	240152	78847	116630
GPT-5-mini	44416	68534	165160	48676	81277
DeepSeek-V3.2	82745	89755	184094	<u>58836</u>	103858
BM25	137230	155033	166033	106051	141087
Qwen3-Emb-4B	133863	172219	188112	113235	151857
GraphRAG	131937	174512	168065	118035	148137
MemAgent	84526	100590	164319	64233	103417
MemoBrain	128844	127004	173458	77045	126588
Mem0	131693	156635	222465	117150	156986
A-MEM	126692	135192	157307	103522	130678
MemOS	109562	123567	194886	81028	127261
MemoryOS	137656	187311	172611	132442	157430
AWM	179134	148277	218708	94064	160046
SkillWeaver	95321	104407	<u>164551</u>	68708	108247
AgentKB	204919	206348	280370	84725	194090
ACE	151680	174171	258134	126804	177697
ReasoningBank	164874	187292	243550	117035	178188
MemEvolve	113777	115031	188065	76205	123270

Table 19: Efficiency comparison on INEP-EXEC under the 64K setting, measured by steps. Lower is better. The best result in each column is bolded and the runner-up is underlined.

Method	Gorilla File System	Vehicle Control	Trading Bots	Travel Booking	Average
Gemini-3-Flash	12.54	13.87	12.15	12.88	12.85
GPT-5-mini	<u>12.62</u>	12.82	10.38	<u>10.30</u>	11.53
DeepSeek-V3.2	15.72	14.32	<u>10.48</u>	10.64	12.79
BM25	16.22	14.46	10.78	11.24	13.18
Qwen3-Emb-4B	16.04	14.64	11.02	11.32	13.26
GraphRAG	16.40	14.40	11.12	11.30	13.30
MemAgent	15.68	13.88	10.70	10.10	<u>12.59</u>
MemoBrain	15.78	<u>13.44</u>	11.04	10.38	<u>12.66</u>
Mem0	16.22	13.92	10.56	10.50	12.80
A-MEM	15.82	14.06	11.04	10.96	12.97
MemOS	15.40	14.16	<u>10.48</u>	10.44	12.62
MemoryOS	16.84	14.68	<u>10.84</u>	10.98	13.34
AWM	23.00	15.00	12.68	10.94	15.41
SkillWeaver	15.28	14.34	11.68	10.98	13.07
AgentKB	28.82	18.06	20.14	12.62	19.91
ACE	19.44	14.48	11.62	11.86	14.35
ReasoningBank	21.30	15.22	11.58	11.88	14.99
MemEvolve	19.22	14.54	12.62	11.18	14.39

Table 20: Efficiency comparison on INEP-EXEC under the 64K setting, measured by token usage. Lower is better. The best result in each column is bolded and the runner-up is underlined.

Method	Gorilla File System	Vehicle Control	Trading Bots	Travel Booking	Average
Gemini-3-Flash	<u>55926</u>	<u>84115</u>	288709	74250	124675
GPT-5-mini	45294	68810	178818	47621	85136
DeepSeek-V3.2	83828	101644	<u>186160</u>	63713	<u>108837</u>
BM25	134793	154066	256800	108267	163482
Qwen3-Emb-4B	131000	179759	300753	111326	180709
GraphRAG	137896	167379	235670	109425	162592
MemAgent	85190	97681	194087	<u>59835</u>	109198
MemoBrain	108767	105150	233231	74542	130422
Mem0	134828	152657	224629	114879	156748
A-MEM	112351	129972	256085	103096	150426
MemOS	96967	123145	225063	82052	131807
MemoryOS	149634	240353	250932	146350	196817
AWM	181790	163050	302457	91774	184768
SkillWeaver	84013	105967	246821	71215	127004
AgentKB	217087	235739	654378	93989	300298
ACE	165695	174011	332247	128942	200224
ReasoningBank	175052	169242	299487	108938	188180
MemEvolve	118371	113117	303296	77040	152956

Table 21: Efficiency comparison on INEP-EXEC under the 128K setting, measured by average steps. Lower is better. The best result in each column is bolded and the runner-up is underlined.

Method	Gorilla File System	Vehicle Control	Trading Bots	Travel Booking	Average
Gemini-3-Flash	13.44	13.55	11.98	13.68	13.15
GPT-5-mini	12.90	12.48	10.52	10.71	11.66
DeepSeek-V3.2	15.90	13.88	10.78	10.64	12.80
BM25	17.04	14.52	10.61	11.18	13.35
Qwen3-Emb-4B	16.28	14.36	10.15	11.20	13.06
GraphRAG	16.72	14.54	10.28	10.90	13.11
MemAgent	15.74	13.68	11.08	10.32	12.71
MemoBrain	16.00	13.96	10.42	10.60	12.74
Mem0	16.20	13.98	10.58	10.86	12.90
A-MEM	16.06	14.28	10.65	11.20	13.06
MemOS	16.02	14.04	10.49	10.80	12.85
MemoryOS	16.04	13.88	9.06	11.68	13.04
AWM	19.88	15.86	10.64	10.56	14.24
SkillWeaver	17.14	14.16	10.94	10.96	13.30
AgentKB	30.48	14.74	14.87	12.70	18.25
ACE	19.00	14.84	11.67	11.58	14.29
ReasoningBank	21.12	15.10	11.42	12.40	15.01
MemEvolve	16.32	14.64	11.53	11.44	13.51

Table 22: Efficiency comparison on INEP-EXEC under the 128K setting, measured by token usage. Lower is better. The best result in each column is bolded and the runner-up is underlined.

Method	Gorilla File System	Vehicle Control	Trading Bots	Travel Booking	Average
Gemini-3-Flash	60136	82076	336793	80528	141437
GPT-5-mini	46875	67273	178668	50309	85959
DeepSeek-V3.2	83920	96226	199725	64076	110987
BM25	141335	162259	261983	105814	167848
Qwen3-Emb-4B	135416	174900	279903	110877	175274
GraphRAG	144339	167429	215260	106440	158367
MemAgent	84212	92239	224752	61541	115686
MemoBrain	131772	131863	197039	81771	135611
Mem0	133638	155550	229999	117347	159133
A-MEM	118105	132459	221201	106099	144080
MemOS	105597	123065	239596	83622	137459
MemoryOS	142679	219548	215285	158213	180424
AWM	136952	163160	224228	75445	149946
SkillWeaver	99893	104069	218444	71885	123573
AgentKB	227553	116884	395484	95346	205974
ACE	157982	177453	326612	126807	196563
ReasoningBank	156512	141968	240103	97710	189712
MemEvolve	95391	117017	280024	78307	140593